

리눅스 시스템 프로그래밍 학습 기록

남궁명수

[3] IPC 와 Pipe.....	2
[01] 익명 파이프(Anonymous Pipe): 부모-자식 간의 단방향 데이터 흐름 제어	4
[01-1] 독립된 메모리 공간을 가진 두 프로세스가 어떻게 데이터를 주고 받는가?	4
[01-2] 시스템 레벨 관찰: pipe() 시스템 콜과 FD 쌍 분석.....	4
[01-3] 이론	7
[01-4] 추가	7
[02] 파이프의 블로킹(Blocking) 특성: 데이터 흐름 제어의 실체	9
[02-1] 데이터가 없거나 가득 찼을 때, 프로세스는 어떤 상태가 되는가?.....	9
[02-2] 시스템 레벨 관찰: 프로세스 상태 전이(Running -> Sleeping) 포착	9
[02-3] 이론	11
[02-4] 추가	12
[03] 표준 입출력 재지정(dup2): 리눅스 파이프라인의 원리 구현	14
[03-1] ls grep 과 같은 셸 명령어의 연결 원리는 무엇인가?	14
[03-2] 시스템 레벨 관찰: 파일 디스크립터 테이블의 교체 과정 분석.....	14
[03-3] 이론	17
[03-4] 추가	18
[04] 양방향 통신 시뮬레이션: 두 개의 파이프로 부모-자식 간 완전한 대화 구현	20
[04-1] 단방향 파이프 하나로 양방향 대화가 가능한가?	20
[04-2] 시스템 레벨 관찰: Full-duplex 대화 시나리오 및 데드락 방지	20
[03-3] 이론	23
[03-4] 추가	25

[4] IPC 와 Pipe

이 파트를 이해하기 위해서는 먼저 “프로세스는 왜 서로 직접 대화할 수 없는 구조인지”부터 정리해야 한다. 리눅스에서 각 프로세스는 기본적으로 완전히 독립된 주소 공간을 가진다. 즉, 하나의 프로세스에서 생성된 변수나 힙 메모리는 다른 프로세스에서 직접 접근할 수 없으며, fork 를 통해 복제된 이후에도 각각의 메모리는 별도로 관리된다. 이 구조는 안정성과 격리를 위해 설계된 것이지만, 동시에 프로세스 간 데이터 공유를 불가능하게 만든다.

이 한계를 해결하기 위해 등장한 개념이 IPC, Inter-Process Communication 이다. IPC 는 프로세스 간 직접적인 메모리 공유를 허용하는 것이 아니라, 커널이 중간에서 데이터를 전달해주는 구조이다. 즉, 프로세스 A 가 직접 프로세스 B 의 메모리에 접근하는 것이 아니라, 프로세스 A 가 커널에 데이터를 전달하고 커널이 이를 다시 프로세스 B 에게 전달하는 방식으로 동작한다. 이 구조에서 커널은 단순한 전달자가 아니라 데이터 저장, 버퍼 관리, 동기화까지 담당하는 중재자 역할을 수행한다.

Pipe 는 이러한 IPC 구조 중 가장 기본적인 형태로, 커널 내부에 생성되는 버퍼를 통해 프로세스 간 데이터를 전달하는 방식이다. 사용자 입장에서 Pipe 는 파일처럼 보이지만 실제로는 커널 메모리에 존재하는 순환 버퍼 구조이며, 프로세스는 이 버퍼를 파일 디스크립터(File Descriptor)를 통해 접근한다.

Pipe 를 생성하면 일반적으로 읽기용 FD 와 쓰기용 FD 가 한 쌍으로 생성되고, 하나의 프로세스는 write 를 통해 데이터를 커널 버퍼로 전달하고 다른 프로세스는 read 를 통해 그 데이터를 가져가는 구조가 된다.

이때 중요한 점은 Pipe 가 메모리 공유 방식이 아니라 FD 기반 접근이라는 점이다. 즉, 프로세스들은 동일한 메모리를 직접 공유하는 것이 아니라, 커널에 제공하는 추상화된 입출력 인터페이스를 통해 간접적으로 연결된다. fork 이후 부모와 자식 프로세스는 동일한 FD 테이블을 복사받기 때문에 Pipe 를 통해 자연스럽게 연결될 수 있다. 일반적으로 부모는 write end 를 사용하고 자식은 read end 를 사용하여 단방향 통신 구조를 형성한다.

Pipe 의 또 다른 핵심 특징은 기본적으로 Blocking 방식으로 동작한다는 점이다. read 는 읽을 데이터가 없으면 대기 상태로 들어가고, write 는 버퍼가 가득 차면 더 이상 진행하지 못하고 대기하게 된다. 이 과정에서 프로세스는 CPU 를 사용하지 않고 sleep 상태로 전환되며, 데이터 조건이 충족되면 다시 실행 상태로 복귀한다. 즉, Pipe 는 단순한 데이터 전달 도구가 아니라 프로세스 실행 흐름을 제어하는 동기화 메커니즘 역할까지 수행한다.

또한 Pipe 는 내부적으로 제한된 크기의 커널 버퍼를 사용하기 때문에 데이터가 지속적으로 쌓이거나 소비되지 않으면 흐름 제어가 발생한다. 이를 back-pressure 라고 하며, 생산자와 소비자의 속도를 자동으로 맞추는 구조를 형성한다. 이 특성 때문에, Pipe 는 단순 통신뿐만 아니라 시스템 전체의 데이터 흐름제어에도 영향을 미친다.

셸에서 사용하는 파이프 연산자, 즉 `ls | grep` 같은 구조는 이러한 Pipe 와 FD 재지정을 기반으로 구현된다. `dup2` 시스템 콜을 통해 표준 입력과 표준 출력을 Pipe 의 FD 로 교체하면, 한 프로세스의 출력이 다른 프로세스의 입력으로 직접 연결되는 구조를 만들 수 있다. 이 과정은 단순한 문법이 아니라 커널 수준에서 FD 테이블을 재구성하는 동작이다.

마지막으로 Pipe 는 기본적으로 단방향 통신 구조이기 때문에 양방향 통신이 필요할 경우 두 개의 Pipe 를 사용해야 한다. 하나는 부모에서 자식으로, 다른 하나는 자식에서 부모로 데이터를 전달하는 방식이다. 이 구조를 통해 full-duplex 통신이 가능해진다.

결론적으로 IPC 와 Pipe 는 단순한 데이터 전달 기능이 아니라, “서로 격리된 프로세스가 커널을 통해 제한적으로 연결되는 구조”를 이해하는 것이 핵심이며, 이 과정에서, FD, 커널 버퍼, Blocking 동작, 그리고 `dup2` 기반의 리다이렉션 개념이 함께 결합되어 동작한다.

[01] 익명 파이프(Anonymous Pipe): 부모-자식 간의 단방향 데이터 흐름 제어

[01-1] 독립된 메모리 공간을 가진 두 프로세스가 어떻게 데이터를 주고 받는가?

- fork()로 나뉜 두 프로세스는 두 변수를 공유하지 않는데, 부모가 쓴 데이터를 자식이 읽으려면 커널이 어떤 도움이 필요한가?

[01-2] 시스템 레벨 관찰: pipe() 시스템 콜과 FD 쌍 분석

코드:

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <sys/types.h>

int main(){
    int pipe_fds[2]; // 파이프용 FD 두 개(0: 읽기, 1: 쓰기)
    char buf[30];
    char *msg = "Hello from Parent";

    if(pipe(pipe_fds) == -1){
        perror("pipe 생성 실패");
        return 1;
    }

    printf("[파이프 생성 완료]. Read FD: %d, Write FD: %d\n", pipe_fds[0], pipe_fds[1]);

    pid_t pid = fork();

    if(pid == 0){
        close(pipe_fds[1]); // 사용하지 않는 쓰기용 FD 닫음

        printf("[자식] 부모로부터 메시지를 기다리는 중..\n");
        read(pipe_fds[0], buf, sizeof(buf));

        printf("[자식] 받은 메시지: %s\n", buf);
        close(pipe_fds[0]);
    }
    else{
        close(pipe_fds[0]);

        printf("[부모] 자식에게 메시지를 보냄\n");
```

```
write(pipe_fds[1], msg, strlen(msg)+1);

close(pipe_fds[1]);
printf("[부모] 전송 완료\n");
}
return 0;
}
```

결과:

(1) 표준 출력은 다음과 같다.

```
[파이프 생성 완료]. Read FD: 3, Write FD: 4
[부모] 자식에게 메시지를 보냄
[부모] 전송 완료
[자식] 부모로부터 메시지를 기다리는 중..
[자식] 받은 메시지: Hello from Parent
```

(2) strace 출력은 다음과 같다.

```
pipe2([3, 4], 0) = 0
```

이 부분은 커널이 읽기용 FD 3 번과 쓰기용 FD 4 번을 생성했다는 의미이다.

이후 fork 는 내부적으로 clone 시스템 콜로 수행된다.

```
clone(...) = 9045
```

9044 는 부모 PID, 9045 는 자식 PID 이다.

부모는 읽기용 FD 를 닫는다.

```
[pid 9044] close(3)
```

자식은 쓰기용 FD 를 닫는다.

```
[pid 9045] close(4)
```

이후 자식 프로세스는 다음 시스템 콜에서 대기 상태로 들어간다.

```
[pid 9045] read(3, ...)
```

이 시점에서는 아직 부모가 데이터를 쓰지 않았기 때문에 자식은 sleep 상태로 들어가 block 된다. 즉, Pipe 는 기본적으로 Blocking I/O 방식으로 동작한다는 사실을 확인할 수 있다.

그 다음 부모가 데이터를 기록한다.

```
[pid 9044] write(4, "Hello from Parent\n", 18) = 18
```

부모가 데이터를 쓰는 순간 커널 버퍼에 데이터가 채워지고, 대기 중이던 자식의 read 가 즉시 깨어난다.

[pid 9045] <... read resumed>"Hello from ParentW0", 30) = 18

즉, Pipe 는 단순 데이터 전달 구조가 아니라, 프로세스 실행 흐름까지 동기화하는 메커니즘이라는 점을 확인할 수 있다.

결과 해석:

이 학습은 서로 완전히 분리된 메모리 공간을 가진 부모 프로세스와 자식 프로세스가, 커널이 제공하는 Pipe 를 통해 데이터를 전달할 수 있다는 점을 직접 확인하는 것이다. 기존의 fork 실험에서는 부모와 자식이 같은 변수 주소를 가지더라도 실제 메모리는 CoW 구조로 분리되어 있었기 때문에 데이터 공유가 불가능했다. 그러나 이번 학습에서는 메모리를 직접 공유하는 대신, 커널 내부에 생성된 파이프 버퍼를 매개체로 사용하여 프로세스 간 통신이 이루어진다.

프로그램이 시작되면 pipe(pipe_fds) 시스템 콜이 실행되며 커널은 읽기용 FD 와 쓰기용 FD 를 생성한다. 실행 결과에서 확인되는 Read FD:3, Write FD: 4 는 각 파이프의 읽기 끝과 쓰기 끝을 의미한다. 리눅스에서는 이미 표준 입력(0), 표준 출력(1), 표준 에러(2)가 사용 중이므로, 새롭게 생성된 FD 는 일반적으로 3 번부터 할당된다.

이후 fork 가 호출되면 부모 프로세스의 FD 테이블이 자식에게 복사된다.

즉, 부모와 자식은 동일한 파이프 FD 를 공유하게 되며, 이를 통해 같은 커널 파이프 버퍼에 접근할 수 있게 된다. 여기서 중요한 점은 부모와 자식이 같은 메모리를 공유하는 것이 아니라, 동일한 커널 객체를 가리키는 FD 를 각각 가지고 있다는 점이다.

부모 프로세스는 읽기용 FD 인 pipe_fds[0]를 닫고 쓰기만 수행한다. 반대로 자식 프로세스는 쓰기용 FD 인 pipe_fds[1]를 닫고 읽기만 수행한다.

사용하지 않는 FD 를 닫는 이유는 단순한 메모리 절약이 아니라, 파이프의 EOF 처리와 데이터 흐름 제어를 정상적으로 동작시키기 위함이다.

만약 쓰기용 FD 가 열려 있는 상태로 남아 있으면 커널은 아직 데이터를 더 쓸 프로세스가 존재한다고 판단하기 때문에 read 가 EOF 를 감지하지 못하고 무한 대기 상태에 빠질 수 있다.

실행 흐름을 보면 부모 프로세스는 write(pipe_fds[1], msg, strlen(msg) + 1)를 통해 “Hello from Parent” 문자열을 커널 파이프 버퍼에 기록한다.

이후 자식 프로세스는 read(pipe_fds[0], buf, sizeof(buf))를 호출하여 해당 데이터를 읽어온다.

이 과정에서 데이터는 부모 메모리에서 자식 메모리로 직접 이동하는 것이 아니라 다음과 같은 경로를 거친다.

부모 프로세스 메모리 -> 커널 파이프 버퍼 -> 자식 프로세스 메모리

즉, 커널이 중간 관리자 역할을 수행하며, 프로세스 간 격리를 유지한 상태에서 안전하게 데이터를 전달하는 구조이다.

[01-3] 이론

이 학습은 IPC의 핵심 구조를 보여준다. Pipe는 익명 파이프(Anonymous Pipe) 형태의 단방향 통신 구조이며, 커널 메모리에 존재하는 버퍼를 통해 데이터가 전달된다. 프로세스 간 직접 메모리 접근은 여전히 금지되어 있지만, 커널이 중재하는 방식으로 제한적인 데이터 흐름이 허용된다.

또한, Pipe는 파일 기반 인터페이스를 따르기 때문에 모든 통신은 File Descriptor를 통해 수행된다. 즉, 리눅스에서는 파일, 소켓, 파이프가 모두 동일한 FD 모델로 추상화되어 있다는 점도 함께 확인할 수 있다.

마지막으로 이번 학습은 셸은 | 연산자 구조를 이해하기 위한 가장 기초적인 단계이다. 이후 `dup2()`를 이용하면 Pipe의 FD를 표준 입력과 출력으로 연결하여 `ls | grep`과 같은 리눅스 파이프라인 구조를 직접 구현할 수 있게 된다.

[01-4] 추가

샌드박스 및 모니터 간의 로그 전달

샌드박스(Sandbox) 구조는 악성코드 분석이나 보안 엔진 설계에서 가장 기본이 되는 격리 실행 모델이다. 핵심 목적은 “의심스러운 프로그램을 실제 시스템과 완전히 분리된 환경에서 실행하면서도, 그 동작은 실시간으로 관찰할 수 있게 하는 것”이다.

이 구조에서 중요한 것은 역할 분리이다.

자식 프로세스는 샌드박스 환경으로 동작하며, 분석 대상 파일이나 의심스러운 실행 코드를 실제로 실행하는 역할만 담당한다. 반면 부모 프로세스는 모니터 또는 보안 엔진으로 동작하며, 자식 프로세스의 실행 결과와 행위를 수집하고 분석하는 역할을 맡는다. 즉, 실행과 감시는 완전히 분리되지만, 동시에 끊김 없이 연결되어야 하는 구조이다.

이때 가장 중요한 연결 통로가 바로 익명 파이프이다.

샌드박스 프로세스는 원칙적으로 화면 출력이나 외부 파일 기록 같은 직접적인 I/O를 제한받는다. 대신 자신이 수행하는 모든 행위, 예를 들어, 파일 열기, 프로세스 생성, 네트워크 접근 시도 등을 문자열 형태의 로그로 만들어 파이프에 기록한다. 이 데이터는 커널 내부 버퍼를 통해 부모 프로세스로 전달된다.

부모 프로세스는 이 파이프의 읽기 끝을 통해 자신의 모든 행위를 실시간으로 수신한다. 중요한 점은 이 과정에서 자식 프로세스가 부모에게 직접 접근하거나 통신 구조를 변경할 수 없다는 것이다. 자식은 단순히 `write()`를 통해 데이터를 흘려보내는 역할만 수행하며, 통신의 흐름과 신뢰성은 커널이 보장한다.

이 구조의 보안적 의미는 매우 크다. 파이프는 사용자 공간이 아니라 커널 공간에서 관리되기 때문에, 자식 프로세스가 악성 동작을 수행하더라도 통신 경로 자체를 조작하거나 차단하는 것은 사실상 불가능하다. 또한, 부모 프로세스는 자식이 어떤 방식으로

동작하더라도 항상 동일한 방식으로 로그를 수신할 수 있기 때문에, 분석의 일관성과 신뢰성이 유지된다.

- 실행 환경과 분석 환경의 완전한 분리
- 커널 기반 통신을 통한 로그 전달의 신뢰성 확보
- 악성 프로세스의 통신 경로 조작 방지
- 실시간 행위 기반 분석 가능

즉, 샌드박스와 모니터 구조에서 파이프는 단순한 데이터 전달 도구가 아니라, “격리된 실행 환경과 분석 시스템을 연결하는 유일한 신뢰 채널” 역할을 수행한다고 볼 수 있다.

[02] 파이프의 블로킹(Blocking) 특성: 데이터 흐름 제어의 실체

[02-1] 데이터가 없거나 가득 찼을 때, 프로세스는 어떤 상태가 되는가?

- 읽을 데이터가 없는 파이프에 read()를 시도하거나, 버퍼가 꽉 찬 파이프에 write()를 시도하면 에러가 나는가 아니면 기다리는가?

[02-2] 시스템 레벨 관찰: 프로세스 상태 전이(Running -> Sleeping) 포착

코드:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main(){
    int pipe_fds[2];
    char buf[100];

    pipe(pipe_fds);

    pid_t pid = fork();

    if(pid == 0){
        close(pipe_fds[0]);

        printf("[자식] 5초 뒤 데이터 전송 ...Wn");
        sleep(5);

        write(pipe_fds[1], "Wake up, Parent", 16);
        printf("[자식] 전송 완료Wn");
        close(pipe_fds[1]);
    }
    else{
        close(pipe_fds[1]);

        printf("[부모] 자식에게 데이터 요청(Read 호출)Wn");

        read(pipe_fds[0], buf, sizeof(buf));

        printf("[부모] 자식에게 받은 데이터: %sWn", buf);
        close(pipe_fds[0]);
    }
}
```

```
    return 0;
}
```

결과:

(1) 표준 출력은 아래와 같다.

```
[부모] 자식에게 데이터 요청(Read 호출)
[자식] 5 초 뒤 데이터 전송 ...
[자식] 전송 완료
[부모] 자식에게 받은 데이터: Wake up, Parent
```

(2) strace 결과:

strace 로그를 통해 부모 프로세스와 자식 프로세스가 파이프를 이용하여 통신하는 과정과 Blocking I/O의 동작을 시스템 콜 수준에서 확인할 수 있다.

먼저 프로그램은 pipe2() 시스템 콜을 통해 파이프를 생성한다.

```
pipe2([3, 4], 0) = 0
```

여기서 파일 디스크립터 3은 읽기(read)용, 4는 쓰기(write)용 파이프 끝을 의미한다.

이후 clone() 호출을 통해 자식 프로세스가 생성된다.

```
clone(child_stack=NULL,
flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD, ...)
```

자식 프로세스는 읽기 끝을 닫고 5 초 동안 sleep 상태에 들어간다.

```
[pid 14643] close(3) = 0
[pid 14643] clock_nanosleep(CLOCK_REALTIME, 0, {tv_sec=5, tv_nsec=0}, ...)
```

반면, 부모 프로세스는 쓰기 끝을 닫은 뒤 read()를 호출한다.

```
[pid 14642] close(4) = 0
[pid 14642] read(3, <unfinished ...>
```

여기서 <unfinished ...>는 read() 시스템 콜이 아직 완료되지 않았음을 의미한다. 즉, 부모 프로세스는 파이프에 데이터가 들어올 때까지 내부에서 대기 상태로 전환된 것이다.

5 초 후 자식 프로세스가 데이터를 기록한다.

```
[pid 14643] write(4, "Wake up, ParentW0", 16 ...)
```

데이터가 파이프에 기록되자 대기 중이던 부모 프로세스의 read()가 다시 실행되며 반환된다.

```
[pid 14642] <... read resumed>"Wake up, ParentW0", 100) = 16
```

이후 부모 프로세스는 전달받은 데이터를 출력하고 프로그램이 종료된다.

결과 해석:

이 학습은 Blocking I/O 의 동작 원리를 이해하기 위한 예이다.

부모 프로세스는 read() 호출 시점에 파이프 내부에 데이터가 존재하지 않기 때문에 즉시 반환되지 못하고 block 상태에 들어가게 된다.

이 과정에서 커널은 부모 프로세스를 실행 대기 큐(wait queue)에 넣고 CPU 를 다른 작업에 양보하도록 만든다.

이후 자식 프로세스가 sleep(5)를 마친 뒤 write()를 수행하여 데이터를 파이프에 기록하면, 커널은 대기 중이던 부모 프로세스를 다시 깨우고 read()를 완료시킨다. 따라서 부모 프로세스는 데이터를 받을 때까지 실제로 실행이 중단된 상태로 유지된다.

또한, 이 구조는 파이프 기반 IPC 에서의 흐름 제어 개념과도 연결된다.

현재 학습에서는 읽는 쪽이 데이터를 기다리는 상황이지만, 반대로 읽는 쪽의 처리 속도가 느려 파이프 버퍼가 가득차게 되면 write()를 수행하는 프로세스 역시 block 상태에 들어갈 수 있다. 즉, 파이프는 읽기와 쓰기 속도를 커널 수준에서 자연스럽게 동기화하는 역할을 수행한다.

[02-3] 이론

Blocking I/O 와 파이프 용량(Pipe Capacity)

Blocking I/O 는 프로세스가 입출력(I/O) 작업을 요청했을 때, 해당 작업을 즉시 수행할 수 없는 경우 커널이 프로세스의 실행을 일시 중단시키는 방식이다.

예를 들어, read() 호출 시 읽을 데이터가 아직 존재하지 않거나, write() 호출 시 데이터를 저장할 공간이 부족한 경우 프로세스는 즉시 반환되지 않고 대기 상태(Blocked State)로 전환된다. 이 과정에서 커널은 해당 프로세스를 Wait Queue 에 등록하고 CPU 를 다른 작업에 할당한다. 이후 데이터가 도착하거나 버퍼 공간이 확보되면 커널이 다시 프로세스를 깨워 I/O 작업을 이어서 수행하도록 만든다. 따라서 Blocking I/O 는 불필요한 반복 검사(Busy Waiting)를 줄이고 CPU 자원을 효율적으로 사용할 수 있도록 해준다.

파이프(Pipe)는 커널 내부 버퍼를 이용하여 프로세스 간 데이터를 전달하는 IPC(Inter-Process Communication) 메커니즘이다.

리눅스의 파이프는 일반적으로 약 64KB 크기의 버퍼를 가지며, 읽는 프로세스와 쓰는 프로세스 사이의 데이터 흐름을 조절하는 역할을 수행한다.

만약 읽는 쪽의 처리 속도가 느려 버퍼가 가득 찬 상태에서 추가적인 write()가 발생하면, 커널은 쓰는 프로세스를 block 상태로 전환시킨다.

반대로 버퍼가 비어 있는 상태에서 read()를 수행하면 읽는 프로세스가 block 상태로 들어가게 된다.

이러한 동작은 Back-pressure(역압) 메커니즘으로 설명할 수 있다. Back-pressure 는 데이터를 소비하는 쪽의 처리 속도에 맞추어 데이터를 생산하는 쪽의 속도를 자연스럽게 제한하는 방식이다.

즉, 소비자가 데이터를 충분히 읽어가지 못하면 생산자는 더 이상 데이터를 밀어 넣지 못하고 대기하게 되며, 이를 통해 시스템 전체의 데이터 흐름이 자동으로 조절된다. 이는 프로세스 간 통신에서 데이터 손실이나 과도한 버퍼 사용을 방지하는 중요한 역할을 한다.

[02-4] 추가

무한 대기를 이용한 서비스 거부(DoS) 위협

파이프 기반 IPC는 구조적으로 매우 안정적인 통신 방식이지만, 그 “Blocking 특성” 자체가 보안 관점에서는 공격 표면이 될 수 있다. 특히 보안 프로그램처럼 외부 프로세스와 지속적으로 통신하며 데이터를 수집하는 구조에서는, 단순한 지연이 전체 시스템 마비로 이어질 수 있다.

대표적인 문제가 Pipe Hang, 즉 무한 대기 상태이다. 파이프에서 read()는 기본적으로 Blocking 방식으로 동작하기 때문에, 읽을 데이터가 도착하지 않으면 해당 스레드는 커널 내부에서 Sleep 상태로 전환된다. 이 상태에서는 CPU를 점유하지 않지만, 해당 스레드는 더 이상 진행되지 못하고 완전히 멈춘 것처럼 동작한다.

이 구조를 악용하면 다음과 같은 상황이 발생할 수 있다.

보안 엔진이 분석을 위해 샌드박스 또는 외부 모듈과 파이프를 통해 연결되어 있다고 가정할 때, 악성 프로세스가 의도적으로 아무 데이터도 보내지 않거나, 파이프 연결만 유지한 채 응답을 지연시키면 보안 엔진의 read() 호출은 계속 블로킹 상태에 머무르게 된다. 만약 이 read()가 메인 스레드 또는 핵심 분석 스레드에서 수행되고 있다면, 해당 스레드 하나의 정지만으로도 전체 보안 시스템이 응답 불가 상태에 빠질 수 있다. 결과적으로 보안 기능 자체가 정지된 것처럼 보이는 서비스 거부 상황이 발생한다.

이 문제의 본질은 “데이터 부재”가 아니라 “종료 조건의 부재”이다.

즉, 시스템이 언제 통신이 끝났는지 판단할 수 없기 때문에 계속해서 데이터를 기다리게 되는 것이다. 따라서 IPC 설계에서는 단순히 데이터를 주고받는 것뿐만 아니라, 통신이 끝났다는 신호를 명확하게 정의하는 것이 필수적이다.

이러한 문제를 해결하기 위한 대표적인 대응 전략은 다음과 같다.

첫째, Non-blocking I/O 방식이다. O_NONBLOCK 플래그를 설정하면 read()는 데이터가 없을 때 영원히 대기하지 않고 즉시 실패하거나 EAGAIN 같은 에러를 반환한다. 이를 통해 스레드가 영구적으로 묶이는 상황을 방지할 수 있다.

둘째, I/O Multiplexing 방식이다.

select, poll, epoll과 같은 메커니즘을 사용하면, 프로세스가 직접 기다리는 구조가 아니라 “데이터가 준비되었을 때만 커널이 알려주는 구조”로 바뀐다. 즉, 능동적으로 기다리는 것이 아니라 이벤트 기반으로 전환되는 것이다.

셋째, Timeout 기반 제어이다. 일정 시간 이상 데이터가 도착하지 않으면 강제로 연결을 종료하거나 예외 처리를 수행하여 시스템이 무한 대기에서 빠지지 않도록 만든다. 이는 보안 엔진처럼 안정성이 중요한 시스템에서 중요한 방법 중 하나이다.

결론적으로 Pipe Hang 문제는 단순한 성능 이슈가 아니라, 시스템 전체를 멈추게 할 수 있는 구조적 취약점이다. 따라서 IPC 설계에서는 “데이터 전달”보다 “언제 기다림을 멈출 것인가”를 함께 설계하는 것이 핵심이다.

[03] 표준 입출력 재지정(dup2): 리눅스 파이프라인의 원리 구현

[03-1] ls | grep 과 같은 셸 명령어의 연결 원리는 무엇인가?

- 한 프로그램의 출력(stdout)을 다른 프로그램의 입력(stdin)으로 어떻게 강제로 연결하는가?

[03-2] 시스템 레벨 관찰: 파일 디스크립터 테이블의 교체 과정 분석

코드:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(){
    int pipe_fds[2];
    pipe(pipe_fds);

    if(fork() == 0){
        // -- 첫 번째 자식: ls -l (데이터를 보내는 쪽) --
        // 1. 자신의 표준 출력 -> pipe_fds[1](쓰기)로 교체
        dup2(pipe_fds[1], STDOUT_FILENO);

        // 2. 교체 후에는 기존 파이프 FD 닫기
        close(pipe_fds[0]);
        close(pipe_fds[1]);

        // 3. printf나 ls의 결과는 화면이 아닌 파이프로 전송
        execlp("ls", "ls", "-l", NULL);
    }

    if(fork() == 0){
        // -- 두 번째 자식: grep .c 실행(데이터를 받는쪽) --
        // 1. 자신의 표준 입력을 파이프 읽기로 교체
        dup2(pipe_fds[0], STDIN_FILENO);

        close(pipe_fds[0]);
        close(pipe_fds[1]);

        // 2. grep은 키보드가 아닌 파이프로부터 데이터를 받음
        execlp("grep", "grep", ".c", NULL);
    }
}
```

```
close(pipe_fds[0]);
close(pipe_fds[1]);
wait(NULL);
wait(NULL);

return 0;
}
```

결과:

(1) 표준 출력 결과는 아래와 같다.

```
-rw-rw-r-- 1 pumpkinbee pumpkinbee 985 May 12 05:42 01_pipe_basic.c
-rw-rw-r-- 1 pumpkinbee pumpkinbee 695 May 12 08:09 02_pipe_blocking.c
-rw-rw-r-- 1 pumpkinbee pumpkinbee 1077 May 12 10:05 03_pipe_redirection.c
```

(2) starce 는 다음과 같다.

먼저 다음 시스템 콜을 통해 파이프가 생성된다.

```
pipe([3, 4]) = 0
```

- 3 번 FD -> 읽기(Read End)

- 4 번 FD -> 쓰기(Write End)

를 의미한다.

이후 fork()가 내부적으로 clone() 시스템 콜로 수행되며 부모 프로세스의 FD 테이블이 자식 프로세스에 복사된다.

가장 중요한 부분은 다음 dup2() 호출이다.

```
dup2(4, 1)
```

이는 1 번이 4 번과 같은 대상을 바라보게 만든다.

반대로 grep 프로세스에서는 다음 호출이 나타난다.

```
dup2(3, 0)
```

이는 0 번이 3 번과 같은 대상을 바라보게 만든다. grep 의 입력이 키보드가 아니라 파이프가 되었음을 나타낸다.

즉, starce 결과를 통해 확인할 수 있는 흐름은 다음과 같다.

```
pipe 생성
→ fork
→ dup2 로 stdout/stdin 교체
→ execve 로 실제 프로그램 실행
→ write/read 를 통한 Pipe 데이터 전달
```

결과 해석:

프로그램 실행 결과로 현재 디렉터리에 존재하는 .c 파일 목록만 출력된 것은, ls -l 명령의 출력 결과가 터미널 환경으로 직접 출력되지 않고 파이프를 통해 grep .c 프로세스의 입력으로 성공적으로 전달되었음을 의미한다.

즉, 사용자는 하나의 명령처럼 보이는 ls | grep 구조가 실제로는 서로 독립된 두 개의 프로세스가 Pipe 와 File Descriptor 재지정을 통해 연결된 결과라는 사실을 확인할 수 있다.

프로그램 내부에서는 먼저 pipe() 시스템 콜을 통해 읽기용 FD 와 쓰기용 FD 가 생성된다. 이후 첫 번째 자식 프로세스는 다음 코드를 수행한다.

```
dup2(pipe_fds[1], STDOUT_FILENO);
```

이 호출은 기존의 표준 출력(stdout, 1 번 FD)이 가리키던 터미널 연결을 제거하고, 대신 파이프의 쓰기용 FD 를 가리키도록 변경한다.

따라서 이후 실행되는 ls -l 은 자신이 화면에 출력한다고 생각하지만, 실제로는 커널의 파이프 버퍼에 데이터를 기록하게 된다.

반대로 두 번째 자식 프로세스에서는 다음 코드가 실행된다.

```
dup2(pipe_fds[0], STDIN_FILENO);
```

이 호출은 표준 입력(stdin, 0 번 FD)을 키보드 입력 대신 파이프 읽기용 FD 로 교체한다. 따라서 grep .c 는 사용자의 입력을 기다리는 것이 아니라, 파이프를 통해 전달된 ls -l 의 출력 데이터를 입력으로 받게 된다.

결과적으로 시스템 내부에서는 다음과 같은 데이터 흐름이 형성된다.

```
ls stdout(1)
→ pipe write FD
→ 커널 파이프 버퍼
→ pipe read FD
→ grep stdin(0)
→ grep stdout(1)
→ 터미널 출력
```

즉, ls 의 출력 파이프를 통해 grep 의 입력으로 연결되고, grep 은 .c 문자열이 포함된 줄만 필터링하여 다시 자신의 stdout 으로 출력한다. grep 의 stdout 은 별도로 변경하지 않았기 때문에 최종 결과만 터미널에 표시된다.

[03-3] 이론

File Descriptor Duplication 및 Redirection

리눅스에서 프로세스는 파일, 터미널, 소켓, 파이프 등을 모두 File Descriptor(FD)라는 정수 번호를 통해 접근한다. 특히 다음 세 개의 표준 FD 는 모든 프로세스에 기본적으로 열려 있다.

- 0 번 FD: 표준 입력(stdin)
- 1 번 FD: 표준 출력(stdout)
- 2 번 FD: 표준 에러(stderr)

일반적인 프로그램은 단순한 stdout(1 번 FD)에 데이터를 출력할 뿐이며, 실제 출력 대상이 화면인지 파일인지 파이프인지는 알지 못한다. 운영체제가 FD 가 가리키는 대상을 결정하기 때문이다.

dup2(oldfd, newfd) 시스템 콜은 newfd 가 기존에 가리키던 연결을 제거하고, 대신 oldfd 와 동일한 커널 객체를 가리키도록 만드는 기능을 수행한다.

즉,

```
dup2(pipe_fds[1], STDOUT_FILENO);
```

는 “stdout 을 터미널이 아닌 Pipe 로 바꿔치기한다”는 의미가 된다.

이 구조 덕분에 리눅스는 프로그램 자체를 수정하지 않고도 데이터 흐름을 자유롭게 변경할 수 있다.

예를 들어,

```
ls > out.txt
```

는 stdout 을 파일로 재지정하는 것이고,

```
ls | grep .c
```

는 stdout 을 Pipe 로 연결하여 다른 프로세스의 stdin 으로 전달하는 구조이다.

즉, 리눅스의 리다이렉션(>)과 파이프라인(|)은 모두 “FD 연결 대상 변경”이라는 동일한 원리 위에서 동작하며, 그 핵심 시스템 콜이 바로 dup2()이다.

[03-4] 추가

(1) dup2()

“출력 통로를 바꿔 끼우는 함수”라고 생각하자.

원래 프로그램은 보통 아래와 같이 동작한다.

```
printf() -> stdout(1 번 FD) -> 터미널 화면
```

즉, printf()는 내부적으로 “1 번 파일 디스크립터(stdout)”에 출력한다.

그런데 dup2()를 사용하면 이 1 번 통로를 다른 대상으로 바꿔버릴 수 있다.

예를 들어,

```
int fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);  
dup2(fd, STDOUT_FILENO);
```

이 코드의 의미는 다음과 같다.

기존:

stdout(1 번) → 터미널

변경 후:

stdout(1 번) → output.txt

즉, 이제부터 printf()는 화면이 아니라 파일에 출력된다.

그래서 아래코드:

```
printf("HelloWn");
```

는 화면에 안 나오고 output.txt 안에 저장된다.

핵심은 dup2(oldfd, newfd)가:

newfd 가 oldfd 를 가리키게 만든다.

여기를 헷갈리면 안됨

예를 들어:

```
dup2(fd, 1);
```

은 1 번(stdout)이 fd 와 같은 곳을 바라보게 만든다는 뜻이다.

(2) 출력 가로채기를 통한 악성 행위 감시

악성코드 분석 환경에서 중요한 전략 중 하나는 “프로세스가 외부로 내보내는 모든 정보를 통제하고 수집하는 것”이다. 그중에서도 가장 단순하면서도 강력한 정보원이 바로 표준 출력과 표준 에러이다. 많은 악성코드는 자신이 수행한 동작의 결과, 감염, 성공 여부, 오류 메시지, 심지어는 C&C(Command and Control) 서버 주소와 같은 정보를 디버깅 메시지 형태로 출력하기도 한다. 따라서 이 출력 채널을 통제하는 것만으로도 상당한 수준의 행위 분석이 가능해진다.

이때 사용되는 핵심 기술이 dup2() 기반의 출력 재지정이다. 보안 엔진은 악성 프로세스를 실행하기 전에 표준 출력과 표준 에러를 원래의 터미널로 두지 않고, 대신 파이프의 쓰기 끝으로 연결한다. 즉, 다음과 같은 구조로 FD를 재구성한다.

표준 출력(1) -> 파이프 write end

표준 에러(2) -> 파이프 write end

이 과정은 dup2() 시스템 콜을 통해 이루어지며, 결과적으로 악성 프로세스는 자신이 여전히 일반 터미널에 출력하고 있다고 인식하지만, 실제 출력은 모두 커널을 통해 파이프로 전달된다.

이 구조는 “프로세스의 인식과 실제 데이터 흐름이 분리된다”는 점이다.

악성코드는 출력이 외부 화면으로 나간다고 생각하지만, 실제로는 모든 문자열이 보안 엔진이 읽는 파이프 버퍼로 들어간다. 보안 엔진은 이 데이터를 실시간으로 수집하여 로그로 저장하거나, 패턴 분석을 통해 악성 행위를 탐지할 수 있다.

이 방식은 단순한 로그 수집을 넘어, 실시간 행위 기반 분석의 기초가 된다.

예를 들어, 특정 문자열 패턴(의심스러운 도메인, 셸 명령어, 권한 상승 시도 등)이 출력되는 순간 즉시 탐지 규칙을 적용할 수 있으며, 경우에 따라서는 프로세스를 즉시 종료하거나 네트워크를 차단하는 대응으로 이어질 수 있다.

보안 관점에서 이 구조가 중요한 이유는 두 가지이다. 첫째, 출력 가로채기는 악성코드가 통신 경로를 명시적으로 열지 않아도 정보를 수집할 수 있는 “비침투적 감시”방식이라는 점이다. 둘째, 이 방식은 커널 수준에서 FD를 재지정하기 때문에, 악성 프로세스가 이를 인지하거나 쉽게 우회하기 어렵다는 점이다.

결과적으로 dup2()를 이용한 출력 가로채기는 단순한 입출력 제어 기법이 아니라, 프로세스의 외부 행동을 강제로 관찰 가능한 형태로 변환하는 핵심 메커니즘이다.

[04] 양방향 통신 시뮬레이션: 두 개의 파이프로 부모-자식 간 완전한 대화 구현

[04-1] 단방향 파이프 하나로 양방향 대화가 가능한가?

- 하나의 파이프에 부모와 자식이 동시에 쓰고 읽으면 어떤 문제가 발생하는가?

[04-2] 시스템 레벨 관찰: Full-duplex 대화 시나리오 및 데드락 방지

코드:

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>

int main(){
    int p2c[2], c2p[2]; // 두 개의 파이프 배열
    char buf[100];

    // 1. 파이프 두 개 생성
    if(pipe(p2c) < 0 || pipe(c2p) < 0) {
        perror("pipe 생성 실패");
        return 1;
    }

    pid_t pid = fork();

    if(pid == 0){
        close(p2c[1]); // 부모꺼 - 부모의 쓰기용 닫음
        close(c2p[0]); // 자식꺼 - 자식의 읽기용 닫음(내꺼 내가 읽을일 없음)

        // 부모의 질문 읽기
        read(p2c[0], buf, sizeof(buf));
        printf("[자식] 질문 받음: %s\n", buf);

        // 부모에게 답변 보내기
        char *answer = "제 이름은 자식 프로세스입니다.";
        write(c2p[1], answer, strlen(answer)+ 1);

        close(p2c[0]);
        close(c2p[1]);
    }
    else {
```

```

        close(p2c[0]); // 부모꺼 - 부모의 읽기용 닫음
        close(c2p[1]); // 자식꺼 - 자식의 쓰기용 닫음

        // 자식에게 질문 보내기
        char *question = "너의 이름은 무엇입니까?";
        write(p2c[1], question, strlen(question)+1);

        // 자식 답변 기다리기
        read(c2p[0], buf, sizeof(buf));
        printf("[부모] 답변 받음: %s\n", buf);

        close(p2c[1]);
        close(c2p[0]);
    }
    return 0;
}

```

결과:

표준 출력 결과는 아래와 같다.

```

[자식] 질문 받음: 너의 이름은 무엇입니까?
[부모] 답변 받음: 제 이름은 자식 프로세스입니다.

```

결과 해석:

실행 결과를 보면 부모 프로세스와 자식 프로세스가 두 개의 파이프를 이용해 정상적으로 양방향 통신을 수행하는 것을 확인할 수 있다.

먼저 부모 프로세스는 p2c[1] 파이프의 쓰기 전용 FD 를 통해 질문 문자열을 전송한다.

```
write(p2c[1], question, strlen(question) + 1);
```

이때 문자열 데이터는 부모 프로세스 메모리에서 자식 프로세스로 직접 이동하는 것이 아니라, 커널 내부의 Pipe Buffer 에 기록된다. 이후 자식 프로세스는:

```
read(p2c[0], buf, sizeof(buf));
```

를 호출하여 해당 버퍼로부터 데이터를 읽어온다.

즉, 파이프는 “프로세스 간 직접 메모리 공유”가 아니라, 커널이 중간 저장소 역할을 수행하는 IPC 구조임을 확인할 수 있다.

또한 read()는 단순히 데이터를 복사만하는 것이 아니라, 읽은 바이트 수만큼 Pipe Buffer 내부 데이터를 소비(consumption)한다. 예를 들어, 부모가 문자열 길이만큼 write()를

수행하면, 커널은 그 크기만큼 버퍼에 저장하고, 자식이 read()를 수행할 때 실제 읽은 바이트 수만큼 버퍼에서 제거한다.

즉, Pipe 는 메시지 단위(Message-based)가 아니라 연속적인 바이트 스트림(Byte Stream) 기반으로 동작한다.

현재 코드에서는

```
write() 1 회  
read() 1 회
```

형태로 단순하게 구성되어 있기 때문에 데이터 경계가 자연스럽게 유지된다.
그러나 실제 IPC 환경에서는 여러 번의 write()가 섞일 수 있기 때문에 프로토콜 설계가 중요하다.

또한, 학습에서는 사용하지 않는 FD 를 명확하게 닫는 과정이 매우 중요하다.

부모 프로세스는:

```
close(p2c[0]);  
close(c2p[1]);
```

를 호출하고

자식 프로세스는:

```
close(p2c[1]);  
close(c2p[0]);
```

를 호출한다.

이 과정을 수행하지 않으면 파이프의 쓰기 끝(write end)이 계속 열린 상태로 유지되기 때문에, read()는 “아직 누군가 데이터를 더 보낼 수 있다”고 판단하여 EOF 를 반환하지 못하고 영원히 블로킹 상태에 빠질 수 있다.

즉, IPC 에서는 “사용하지 않는 FD 를 닫는 행위”자체가 통신 종료 신호 역할까지 수행한다.
결과적으로 다음 세 가지를 시스템 레벨에서 확인한 것이다.

1. 파이프는 커널 버퍼를 이용한 IPC 구조이다.
2. 파이프는 단방향 스트림 기반 통신이다.
3. 양방향 통신을 위해서는 두 개의 파이프와 정확한 FD 관리가 필요하다.

[03-3] 이론

IPC 설계와 파이프 클로징(Closing)의 중요성

파이프(Pipe)는 리눅스에서 가장 기본적인 IPC(Inter-Process Communication) 방식 중 하나이며, 서로 독립된 프로세스가 데이터를 주고 받기 위해 사용하는 커널 기반 통신 통로이다.

이전까지의 프로세스는 각자 완전히 분리된 주소 공간을 사용했기 때문에 직접적으로 메모리를 공유할 수 없었지만, 파이프를 사용하면 커널 내부에 생성된 버퍼를 매개체로 하여 안전하게 데이터를 전달할 수 있다.

하지만 파이프의 가장 중요한 특징은 “기본적으로 단방향 통신 구조”라는 점이다.

하나의 파이프는 오직 한 방향으로만 데이터가 흐른다. 즉, 한쪽은 쓰기를 담당하고 다른 한쪽은 읽기를 담당한다. 이러한 구조는 물이 흐르는 수도관과 유사하다.

물은 한 방향으로만 흐를 수 있으며, 동시에 반대 방향으로 흐르지는 않는다.

따라서 하나의 파이프만으로는 완전한 대화 구조를 만들 수 없다.

부모가 자식에게 데이터를 보내는 것은 가능하지만, 같은 파이프를 이용해 자식이 다시 부모에게 자연스럽게 응답하기는 어렵다.

이론적으로는 하나의 파이프에서 읽기와 쓰기를 모두 수행할 수는 없지만, 실제로는 데이터 흐름이 꼬이거나 자신이 쓴 데이터를 자신이 다시 읽는 상황이 발생할 수 있기 때문에 안정적인 양방향 통신 구조로 사용되지 않는다.

이 때문에 리눅스 IPC에서는 보통 다음과 같이 두 개의 파이프를 사용한다.

부모 → 자식 전송용 파이프 자식 → 부모 응답용 파이프

즉, 각각의 데이터 흐름을 독립적으로 분리하여 관리하는 것이다.

이러한 구조를 Full-duplex(전이중)통신이라고 한다.

전화 통화처럼 양쪽이 서로 데이터를 주고받을 수 있는 구조이다. 반면, 하나의 파이프만 사용하는 경우는 Half-duplex(반이중) 통신에 가까운 구조라고 볼 수 있다.

이번 코드에서

<pre>int p2c[2]; int c2p[2];</pre>
--

를 따로 생성한 이유도 바로 이 때문이다.

하나를 부모가 질문을 보내기 위한 전용 통로이고, 다른 하나는 자식이 답변을 보내기 위한 전용 통로이다. 즉, 리눅스의 파이프라인 구조는 단순히 “문자열 전달”이 아니라, 데이터의 흐름 방향 자체를 설계하는 과정이라고 볼 수 있다.

또한, IPC 설계에서 매우 중요한 원칙 중 하나는 “사용하지 않는 파일 디스크립터(FD)를 반드시 닫아야 한다”는 점이다. 이 부분은 단순한 메모리 정리가 아니라 시스템의 동작 자체를 결정하는 핵심 요소이다.

파이프에서 read() 시스템 콜은 기본적으로 Blocking 방식으로 동작한다. 즉, 읽을 데이터가 없으면 CPU 를 계속 사용하는 것이 아니라 커널 내부에서 Sleep 상태로 대기한다. 그리고 다음 두 상황 중 하나가 발생할 때 다시 깨어난다.

첫 번째는 새로운 데이터가 도착했을 때이다.
두 번째는 파이프의 모든 쓰기용 FD 가 닫혀 EOF(End Of File)가 발생했을 때이다.

여기서 핵심은 EOF 조건이다.
커널은 단순히 “현재 데이터가 없는 상태”만 보고 통신 종료를 판단하지 않는다.
커널은 “앞으로 데이터를 더 보낼 가능성이 완전히 사라졌는가?”를 기준으로 EOF 를 판단한다.
즉, 어떤 프로세스라도 해당 파이프의 쓰기용 FD 를 하나라도 열어두고 있으면, 커널은 아직 누군가 데이터를 보낼 가능성이 있다고 생각한다.

그 결과 read()는 다음과 같은 상태에 빠질 수 있다.

읽을 데이터 없음
+
쓰기 FD 아직 열려 있음
=
계속 대기(Block)

즉, 프로세스는 영원히 read()에서 빠져나오지 못하고 무한 대기 상태에 들어간다.
이것이 IPC 설계에서 가장 흔하게 발생하는 실수 중 하나이다.

예를 들어, 부모 프로세스가 이미 데이터 전송을 끝냈더라도 실수로 쓰기용 FD 를 닫지 않았다면 자식 프로세스는

“언젠가 데이터가 더 올지도 모른다”

고 판단하여 계속 기다리게 된다. 이 현상은 CPU 를 사용하는 Busy Waiting 이 아니라 커널 내부의 Sleep 상태로 들어가기 때문에 겉으로 보면 프로그램이 멈춘 것처럼 보인다.

실제로 서버 프로그래밍이나 보안 엔진에서는 이러한 FD 관리 실수가 치명적인 문제를 만든다.

예를 들어, 샌드박스 프로세스가 분석 결과를 전송한 뒤 종료되었는데도 일부 쓰기 FD 가 닫히지 않으면, 메인 엔진은 결과를 기다리며 영원히 블로킹될 수 있다.

즉, IPC 설계에서는 단순히 데이터를 보내는 것보다 “언제 통신이 끝났는지 명확하게 알려주는 것”이 훨씬 중요하다.

[03-4] 추가

커맨드 및 제어(C&C) 통신 구조의 이해

C&C(Command and Control) 통신 구조는 악성코드가 감염 이후에도 외부 공격자와 지속적으로 연결을 유지하면서 원격으로 제어받는 핵심 통신 방식이다.

이 구조를 이해한다는 것은 단순히 네트워크 통신을 보는 것이 아니라, “공격자가 감염된 시스템을 어떻게 조정하는가”를 시스템 레벨에서 분석하는 것이다.

대표적인 형태가 Reverse Shell 이다. 일반적인 클라이언트-서버 구조는 내부 시스템에서 외부 서버에 요청을 보내는 형태지만, Reverse Shell 은 그 흐름이 반대로 뒤집힌다.

감염된 시스템 내부의 악성코드가 외부 공격자의 서버로 먼저 연결을 시도하고, 이후 해당 연결을 통해 셸 입력과 출력이 그대로 전달되는 구조를 만든다.

결과적으로 공격자는 마치 해당 시스템의 터미널을 직접 사용하는 것처럼 명령을 실행할 수 있게 된다.

C&C 구조도 본질적으로 이와 동일한 원리를 확장한 형태이다. 공격자는 제어 서버를 통해 감염된 프로세스에게 명령을 전송하고, 해당 프로세스는 그 명령을 실행한 뒤 결과를 다시 동일한 통신 서버 채널로 반환한다. 이때 내부 구현은 파이프, 소켓, 혹은 혼합된 IPC 구조를 사용할 수 있지만 핵심은 항상 “명령 전달과 반환이 양방향으로 지속되는 구조”라는 점이다.

이러한 구조가 위험한 이유는, 감염된 시스템 내부에서 일반적인 프로그램처럼 보이는 프로세스가 실제로는 외부와 지속적으로 통신하며 제어되고 있기 때문이다. 특히 파이프나 소켓 기반 구조는 커널을 통해 통신이 이루어지기 때문에, 단순한 프로세스 관찰만으로는 정상 프로그램과 구분하기 어렵다.

따라서 탐지 관점에서는 통신 구조 자체의 “패턴”이 중요한 단서가 된다.

예를 들어, 시스템 내부에서 두 프로세스가 비정상적으로 지속적인 양방향 통신을 유지하거나, 짧은 주기로 명령-응답 구조가 반복된다면 이는 정상적인 IPC 라기보다 외부 제어 구조일 가능성이 높다. 또한 명령 전달과 결합 반환이 일정한 규칙 없이 반복되거나, 특정 프로세스가 지속적으로 외부 연결을 유지한 채 입력을 기다리는 구조라면 C&C 채널일 가능성을 의심해야 한다.

결국 C&C 통신 구조는 단순한 IPC 응용이 아니라, “프로세스 간 통신을 이용한 원격 제어 구조”이다. 이 구조를 이해하는 것은 악성코드 분석뿐 아니라, 시스템 내부에서 발생하는 비정상적인 통신 흐름을 식별하고 차단하는 데 핵심적인 기반 지식이 된다.